
Équipe 101

Poly Quiz
Plan de projet

Version 1.9

Historique des révisions

Date	Version	Description	Auteur
2024-09-18	1.0	La rédaction initiale du document plan de projet ainsi que l'introduction	Mehdi El Harami
2024-09-18	1.1	Rédaction de la section 2.1	Mehdi El Harami
2024-09-20	1.2	Rédaction de la section 2.2 et de la section 2.3	Mehdi El Harami
2024-09-21	1.3	Rédaction de la section 3.1 et de la section 3.3	Mehdi El Harami
2024-09-22	1.4	Rédaction de la section 4.1	Mehdi El Harami
2024-09-23	1.5	Rédaction de la section 4.2 et de la section 5	Mehdi El Harami
2024-09-24	1.6	Rédaction de la section 6	Mehdi El Harami
2024-09-24	1.7	Rédaction de la section 3.2	Arman Lidder
2024-09-24	1.8	Rédaction de la section 3.4	Hamza Boukaftane
2024-09-25	1.9	Mise à jour de la section 4.2 et révision final	Hamza Boukaftane

Table des matières

1. Introduction	4
2. Énoncé des travaux	4
2.1. Solution proposée	4
2.2. Hypothèses et contraintes	6
2.3. Biens livrables du projet	7
3. Gestion et suivi de l'avancement	7
3.1. Gestion des exigences	7
3.2. Contrôle de la qualité	8
3.3. Gestion de risque	8
3.4. Gestion de configuration	11
4. Échéancier du projet	12
4.1 Explication du calcul des heures:	12
4.2 Échéancier du projet:	13
5. Équipe de développement	16
5.1. Mehdi El Harami:	16
5.2 Arman Lidder:	16
5.3. Hamza Boukaftane:	16
5.4 Maxime Pageot:	17
5.5 Kasra Rahimi:	17
5.6. Mélissa Hammache:	17
6. Entente contractuelle proposée	17
6.1 Type de contrat	18
6.2 Respecter les échéances intermédiaires	18
6.3 Respect l'échéancier final	18
6.4 Paiement	18
6.5 La négociation de tout changement	18

Plan de projet

1. Introduction

Ce document fait état du plan de projet pour le logiciel Poly Quiz de l'équipe 101. La section 2 aborde la solution proposée au projet, les hypothèses et contraintes liées au projet ainsi que la liste des artéfacts qui devront être créés durant le projet avec leurs dates prévues de livraison. Par la suite, la section 3 présente la gestion des exigences, de la qualité, du risque, et de la configuration. Ensuite la section 4 met en lumière l'échéancier du projet. Dans la section 5, une brève description des membres de l'équipe de développement. Finalement, la section 6 contient l'entente contractuelle proposée.

2. Énoncé des travaux

2.1. Solution proposée

Notre solution proposée est la livraison de PolyQuiz, une plateforme de jeux-questionnaire multimode. L'application sera disponible sous la forme d'un client léger Android fonctionnant sur une tablette (Galaxy Tab S6 Lite) et d'un client lourd fonctionnant sur un appareil Windows. Dans le cas du client lourd, l'application offrira un système de création et d'authentification de compte, un système de clavardage dans lequel il sera possible d'accéder à l'interface de messagerie. Une fonctionnalité de canaux de discussion sera également disponible, où l'on pourra créer, rejoindre et supprimer des canaux, avec un canal principal regroupant tous les utilisateurs. Lors d'une partie, un canal sera créé automatiquement, regroupant tous les joueurs, et sera supprimé à la fin de la partie. Un utilisateur devra pouvoir voir l'historique complet du clavardage de n'importe quel canal. Il y aura également un mode de jeu classique, permettant aux utilisateurs de rejoindre ou de créer des parties en ligne. Ils pourront consulter les parties en attente, voir le nombre de joueurs et le questionnaire choisi. En entrant un code unique, ils pourront accéder directement à une partie spécifique. L'organisateur pourra verrouiller la partie et la démarrer dès qu'au moins un joueur sera présent. Pendant le jeu, les joueurs répondront à des questions, accumulent des points, et un classement final sera affiché à la fin. Un mode de jeu en équipe sera également disponible, dans lequel les joueurs joueront par équipes de deux. Le score d'une équipe sera la somme des points de ses membres, et l'équipe gagnante sera celle qui aura le plus de points. Chaque équipe disposera aussi d'un canal de discussion privé durant la partie. Pour les utilisateurs connectés, l'accès à une interface d'administration, sans mot de passe, leur permettra de créer, modifier ou supprimer des questionnaires. Chaque questionnaire contiendra un nom, une description, un temps de réponse, et des questions de différents types : QRL (réponse libre), QCM (choix multiple), et QRE (réponse estimée). Les questionnaires devront être validés avant d'être sauvegardés et pourront être exportés et importés au format JSON. L'utilisateur pourra également ajouter des images aux questions. Les questionnaires pourront être publics ou privés. Par défaut, ils seront publics, modifiables par tous, et des parties pourront être organisées avec eux. Un questionnaire privé ne sera visible et modifiable que par son créateur, qui pourra le rendre public s'il le souhaite. Il y aura une monnaie virtuelle. À la fin d'une partie, le gagnant

recevra de la monnaie, tandis que les autres joueurs obtiendront un prix de consolation. La monnaie gagnée sera visible dans l'interface du joueur et pourra être utilisée pour acheter des avatars ou des éléments cosmétiques. Ces achats seront liés au compte de l'utilisateur. Un organisateur pourra ajouter un prix d'entrée pour une partie, que les joueurs devront payer pour rejoindre. À la fin, le gagnant remportera $\frac{2}{3}$ des entrées, et les perdants se partageront $\frac{1}{3}$. Si un joueur abandonne, il perd son entrée et ne reçoit pas de lot de consolation. Un utilisateur pourra rejoindre une partie en cours en tant qu'observateur. Il verra la liste des parties disponibles avec un indicateur du nombre d'observateurs. L'utilisateur pourra choisir un thème visuel parmi une liste prédéfinie, qui s'appliquera uniquement à ses interfaces. Le système sauvegardera les choix de thème et de langue pour qu'ils restent actifs après reconnexion, tout en permettant de les modifier ultérieurement. L'utilisateur pourra envoyer, recevoir, accepter ou refuser des demandes d'amis, même si l'autre utilisateur est hors ligne. Il pourra également retirer des amis de sa liste, et celle-ci sera sauvegardée chaque session. La recherche de profils d'utilisateurs se mettra à jour en temps réel et exclura les amis déjà ajoutés. De plus, l'utilisateur pourra créer des parties réservées uniquement à ses amis, accessibles uniquement aux membres de sa liste d'amis.

Le client léger présentera des fonctionnalités l'entièreté des fonctionnalités du client lourd. Cependant, la vue d'administration et de création ne sera pas disponible sur le client léger. De plus, le client léger devra intégrer un système de notification sonore et visuel à la réception de nouveaux messages. Également, un mode de jeu hors ligne sera disponible sur l'interface mobile permettant à l'utilisateur de jouer à des quiz QCM en mode classique lorsqu'il n'est pas connecté à Internet. Il sera donc en mesure de télécharger les quiz préalablement lorsqu'il est connecté. Enfin, la plateforme mobile pourra faire émission de son lorsque le compte à rebours atteint 5 secondes ou encore lorsque le mode panique est activé. De plus, l'utilisateur sur la plateforme mobile sera en mesure d'activer un assistant vocal qui lit les questions durant le jeu de vive voix.

Ci-dessous est le tableau contenant les exigences fonctionnelles essentielles pour le client lourd et le client léger.

Tableau I : Liste des exigences fonctionnelles essentielles pour le client lourd et le client léger

Exigences fonctionnelles essentielles	
Client lourd	Client léger
Clavardage - Intégration	Clavardage - Intégration
Clavardage - Canaux de discussion	Clavardage - Canaux de discussion
Compte utilisateur et historique	Compte utilisateur et historique
Avatar	Avatar
Modes de jeu en équipe et classique	Modes de jeu
Vues d'administration et Vue de création	Système de monnaie virtuelle (partie avec prix)
Système de monnaie virtuelle (partie avec prix)	Observateurs (observé partie en cours)
Observateurs (observé partie en cours)	Personnalisation de l'application (persistance)

Personnalisation de l'application (persistance)	Système d'amis (création d'amis seulement)
Système d'amis (création d'amis seulement)	Mode Hors-ligne
	Lecture vocale des questions

Ci-dessous est le tableau contenant les exigences fonctionnelles souhaitables pour le client lourd et le client léger.

Tableau II : liste des exigences fonctionnelles souhaitables pour le client lourd et le client léger

Exigences fonctionnelles souhaitables	
Client lourd	Client léger
Compte utilisateur et historique	Compte utilisateur et historique
Modes de jeu	Modes de jeu (Élimination)
Système de monnaie virtuelle	Système de monnaie virtuelle (Défis de partie et indices)
Système d'amis	Système d'amis (système de blocage)
Achievements	Achievements
IA correcteur des QRLS	IA correcteur des QRLS
Niveau/expérience	Niveau/expérience
Message audio/images	Message audio/images
Classement général	Classement général
	Son de la tablette
	Soundboard

2.2. Hypothèses et contraintes

2.2.1 Ressources humaines

L'équipe de développement est composée de six membres, chacun disposant d'une disponibilité moyenne de 10 heures par semaine à consacrer au projet. Ces membres possèdent des compétences variées et complémentaires, incluant le développement mobile avec Flutter, le développement backend avec Node.js et MongoDB, la conception d'une application de bureau avec Angular, la gestion de projet, et l'assurance qualité. Chaque membre est responsable d'une partie spécifique du projet, bien qu'une flexibilité soit prévue pour permettre le partage des tâches en fonction des besoins. Cette organisation doit permettre à l'équipe de couvrir efficacement toutes les étapes du projet, malgré les contraintes de temps limité et la disponibilité partielle de chaque membre.

2.2.2 Équipement

En ce qui concerne l'équipement, l'équipe dispose de ressources limitées. Deux membres possèdent des ordinateurs MacBook, tandis que les autres membres utilisent des ordinateurs fonctionnant sous Windows. Pour les tests sur

mobile, l'équipe ne dispose que d'une seule tablette Android, une Galaxy Tab S6 Lite, qui doit être partagée entre tous les membres pour le développement, les tests, et la démonstration du client léger. Bien que l'accès à des simulateurs et émulateurs soit possible, les tests sur un appareil physique sont restreints, ce qui pourrait engendrer des limitations dans la validation de la version mobile de l'application. Ces contraintes matérielles devront être prises en compte dans la planification des phases de développement et de tests.

2.2.3 Échéancier

Le projet a débuté le 26 août 2024, date à laquelle l'équipe a reçu la proposition de projet. L'échéancier est serré, avec des dates clés à respecter. La réponse à l'appel d'offres, incluant la description détaillée de la solution proposée, les échéances, les ressources allouées, doit être soumise au plus tard le 27 septembre 2024. Ensuite, la phase de conception et de développement s'étendra du 1er octobre au 15 novembre 2024. Durant cette période, l'équipe se concentrera sur la conception des interfaces, le développement des fonctionnalités principales et les tests initiaux. Un premier prototype fonctionnel devra être prêt pour une présentation au client prévue le 12 novembre 2024. La phase suivante, dédiée aux tests finaux et aux ajustements, se déroule du 16 au 25 novembre 2024. Elle comprendra des tests de validation, des ajustements en fonction des retours du client, ainsi que des corrections de bogues. Enfin, le produit final, incluant le code source, la documentation et la version installable du logiciel pour les clients léger et lourd, devra être livré le 29 novembre 2024. Cet échéancier exigeant nécessite une gestion rigoureuse du temps et des ressources pour assurer la qualité du produit tout en respectant les délais imposés.

2.3. Biens livrables du projet

- **27 septembre 2024** : Réponse à l'appel d'offres soit le plan de projet, le SRS, la liste d'exigences, le document d'architecture logicielle, le protocole de communication, ainsi que les prototypes de communication client lourd/léger-serveur.
- **29 novembre 2024**: Mise à jour des artefacts remis lors de la remise précédente, le plan de tests, les résultats de tests, ainsi que le code source et exécutable du produit final.

3. Gestion et suivi de l'avancement

3.1. Gestion des exigences

Les exigences du produit final ont été initialement définies dans le document de spécification des exigences (SRS). Ce document détaille de manière exhaustive l'ensemble des fonctionnalités que l'équipe de développement doit implémenter pour répondre aux attentes du client. Afin de s'assurer que ces exigences sont bien comprises et respectées tout au long du projet, l'équipe organisera trois réunions hebdomadaires, les lundi, mercredi et dimanche. Ces rencontres régulières permettront de faire le point sur l'avancement des travaux, de discuter des éventuels obstacles et d'ajuster les priorités en fonction de l'évolution du projet. Le suivi et la gestion de l'ensemble des exigences seront

centralisés dans l'outil de gestion de projet GitLab. Chaque exigence sera fragmentée en différentes demandes, appelées « issues », pour faciliter la gestion et la traçabilité. Après chaque réunion, si l'équipe décide de modifier une exigence, la demande GitLab correspondante sera mise à jour en conséquence. Cette modification sera ensuite répercutée dans le document SRS, afin de maintenir une cohérence entre les spécifications documentées et le travail en cours. Toute modification des exigences sera communiquée au client pour obtenir son approbation. L'équipe prendra soin de l'informer des raisons justifiant ce changement et de ses implications sur le projet. Ce processus d'approbation garantira que toutes les parties prenantes sont alignées et que le produit final réponde bien aux besoins du client.

3.2. Contrôle de la qualité

La rédaction des artefacts sera faite sur Google Docs. Chaque membre aura comme responsabilité de lire tous les autres artefacts sur lesquels il n'a pas travaillé et puis laisser des commentaires pertinents afin que tout le monde puisse les consulter. Un membre de l'équipe sera également responsable de vérifier les fautes d'orthographe à l'aide d'Antidote. Les fonctionnalités de l'application devront être faites en utilisant les branches Git. L'équipe adoptera un mode de travail basé sur les *pull requests*. Avant de fusionner un changement dans la branche principale, un *pull request* sera créé pour discuter et évaluer les modifications avec les autres membres de l'équipe. Au moins deux personnes devront réviser le code avant que le pull request puisse être approuvé. Nous utiliserons également les pipelines d'automatisation de Gitlab pour exécuter la suite de tests ainsi qu'un *linter* à chaque *pull request*. Le *linter* permettra de détecter les erreurs de syntaxe et les incohérences de style dans le code. Ainsi, les développeurs devront effectuer des tests unitaires tout au long du développement du projet. Si des tests échouent ou si le linter détecte des problèmes, la pull request sera bloquée tant qu'une version corrigée sans erreurs n'aura pas été poussée.

3.3. Gestion de risque

La description des risques suit la convention suivante :

- Ampleur : sur une échelle de 1 à 10, 10 étant le risque le plus élevé. Cette analyse est basée sur la probabilité d'occurrence du risque, ainsi que ses impacts.
- Description : une description textuelle du risque ainsi que les problèmes attendus.
- Impact : échelle définissant la portée du risque
 - C – critique (affecte le projet en entier)
 - E – élevé (affecte les fonctionnalités principales du système)
 - M – moyen (devrait être maîtrisable en appliquant une stratégie d'atténuation adéquate)
 - F – faible (l'acceptation du risque est une stratégie envisageable)
- Facteurs : aspects (métriques) du système pouvant être compromis.

- Stratégie de gestion : mesures à prendre afin de gérer le risque

1 - Disponibilité des ressources				
Ampleur	Description	Impact	Facteurs	Stratégie de gestion
5	Étant donné que nous sommes tous étudiants et que certains d'entre nous ont également des engagements professionnels, il est possible que certains membres de l'équipe ne soient pas en mesure de consacrer autant d'heures au projet que prévu. Il peut également arriver que des membres soient dans l'incapacité d'assister à certaines réunions. Si cela se produit, cela risque de ralentir l'avancement du projet et de perturber la planification.	M	Nombre d'heures travaillées Et Rythme/Rapidité du travail	Chaque membre de l'équipe doit faire preuve de discipline en s'organisant de manière efficace pour progresser sur ses tâches, tout en prévoyant ses absences, retards et tout élément pouvant impacter sa performance dans le cadre du projet. En cas d'absences répétées, une discussion sera tenue au sein de l'équipe, et un retour sera communiqué au professeur à ce sujet.

2 - Temps de travail plus élevé que planifié				
Ampleur	Description	Impact	Facteurs	Stratégie de gestion
7	Étant donné que nous avons peu d'expérience dans la planification d'un projet de cette envergure et dans l'utilisation de l'outil Flutter pour le développement mobile, et que nous ne pouvons pas anticiper les imprévus, il est probable que le développement de certaines fonctionnalités prenne plus de temps que prévu. Cela pourrait entraîner des difficultés dans la planification des sprints et affecter la qualité du livrable.	M	Complexité des fonctionnalités à développer	Organiser des sessions de formation cela peut inclure des ateliers, des tutoriels en ligne et mettre en place des réunions hebdomadaires de suivi où chaque membre peut partager ses progrès, ses défis et demander de l'aide si nécessaire.

3 - Mauvaise communication				
Ampleur	Description	Impact	Facteurs	Stratégie de gestion
6	Une communication insuffisante entre les membres de l'équipe et le client peut survenir. Par communication insuffisante, nous entendons des échanges manquants, incomplets ou peu clairs. Cela peut entraîner une mauvaise compréhension de certains sujets.	F	Manque de retour d'information	Établir des canaux de communication appropriés "Discord et Gitlab", et des réunions régulières (hebdomadaires) et des mises à jour, afin d'assurer un flux d'informations constant et transparent.

4- Erreurs dans le design de l'architecture				
Ampleur	Description	Impact	Facteurs	Stratégie de gestion
7	Des erreurs dans le design de l'architecture peuvent causer des problèmes tout au long du projet.	M	Nombre d'erreurs Et Efficacité au travail	Le document d'architecture sera examiné et compris par toute l'équipe avant le début du développement. En cas d'erreurs, celles-ci seront discutées en groupe et corrigées.

5 - Mauvaise planification				
Ampleur	Description	Impact	Facteurs	Stratégie de gestion
5	Une mauvaise planification des sprints et des exigences peut survenir avant et pendant le développement. Étant donné que nous ne pouvons pas toujours évaluer avec précision la difficulté d'une fonctionnalité ni le temps nécessaire à son développement, il est presque impossible d'atteindre une planification parfaite du projet.	M	Complexité des fonctionnalités Et Manque d'expérience de l'équipe	Une planification initiale où les exigences sont clairement documentées et évaluées en équipe. Il faut établir des estimations de temps réalistes, en tenant compte des complexités. Des réunions régulières permettent d'ajuster les priorités et de résoudre les problèmes au fur et à mesure.

6 - Panne de la base de données				
Ampleur	Description	Impact	Facteurs	Stratégie de gestion
2	La base de données connectée au serveur est temporairement indisponible.	C	Interruptions de réseau Et Forte demande sur le serveur	Mettre en place des outils de monitoring pour détecter rapidement les problèmes de performance ou de connectivité et intervenir rapidement. De plus, en cas d'interruption, informer rapidement l'équipe.

7 - Performance des communications client lourd-léger				
Ampleur	Description	Impact	Facteurs	Stratégie de gestion
1	La performance des communications (transmission d'informations, chat, etc.) entre le client lourd et le client léger n'est pas optimale..	C	faible bande passante Et Erreurs dans la configuration des protocoles de communication	Effectuer des tests réguliers pour identifier les goulots d'étranglement. Ensuite, il sera possible d'analyser et d'améliorer la capacité du réseau afin de garantir une bande passante suffisante, tout en s'assurant que les deux clients sont à jour.

3.4. Gestion de configuration

Pour ce qui est du code source, la gestion de configuration sera complètement implémentée à l'aide de l'outil Git, un système de contrôle de version, et d'un répertoire de projet sur GitLab contenant le code source du client lourd, du client léger, du serveur et la documentation. En effet, pour la soumission de changements du code source ou de changements dans la documentation, les changements qui seront poussés sur le répertoire devront être accompagnés d'un titre explicitant le but du changement et une description complète du changement apporté. Chaque changement doit être poussé sur une branche du répertoire associée à la fonctionnalité implémentée. Ainsi, la branche *main* sera protégée des modifications portant des défaillances et fournira une version fonctionnelle. De plus, lors de la fusion d'une branche portant une nouvelle fonctionnalité vers la branche *main*, il faudra effectuer un *merge request* avec une

explication détaillée des changements portés, il faudra également l'approbation de deux membres de l'équipe qui n'ont pas participé au développement de la fonctionnalité et le répertoire GitLab exécutera également les tests unitaires et les tests de *lint* avant d'effectuer la fusion des branches. Si une erreur survient durant les tests, la fusion sera abandonnée. Cela permet un très bon contrôle de l'assurance qualité de la branche *main* et . Également, l'équipe fait l'usage d'un serveur Discord qui intègre un robot communiquant tous les événements qui arrive sur le répertoire GitLab assurant la bonne coordination de l'équipe.

En cas de problème quelconque, l'équipe va évaluer le problème lors d'une des trois rencontres hebdomadaires. Lors de cette rencontre, une nouvelle *issue* sera initiée dans GitLab avec un nom logique et une description explicite du problème et de la solution. Cela assure une bonne traçabilité des problèmes. Lors de l'analyse, l'équipe déterminera les impacts du problème sur l'échéancier du projet et la façon de réallouer les ressources disponibles afin de résoudre le problème en affectant le moins que possible le déroulement du projet. Ainsi, ce processus de gestion des problèmes va assurer une stratégie cohérente et agile de résolution de problème. Également, tous les changements de stratégie seront indiqués dans la documentation.

Enfin, le nom des nouvelles branches devra correspondre au nom de la fonctionnalité dans le document de SRS et la cible de son implémentation, c'est-à-dire soit client léger, client lourd ou serveur. Toutes les commits poussées vers le répertoire du projet sont automatiquement numérotés et archivés ce qui permet d'avoir un historique des versions complètes. En ce qui concerne les modifications dans les artefacts du projet, chaque modification sera enregistrée à la deuxième page des documents en incrémentant le numéro de version de 0.1, en indiquant la date de la modification, en indiquant le nom de l'auteur et un titre indicatif de la modification effectuée. La version initiale est 1.0 pour l'ensemble des artefacts. Cela assure une bonne gestion des configurations.

4. Échéancier du projet

4.1 Explication du calcul des heures:

Tout d'abord, nous avons calculé le nombre d'heures de travail disponibles jusqu'à la remise du projet le 29 novembre. Le calcul est le suivant :

9 semaines x 6 personnes x 10 heures/semaine/personne = 540 heures. (Il s'agit du total des heures pour les 9 semaines de développement, sans prendre en compte une estimation plus réaliste ni inclure les heures dédiées à la réponse à l'appel d'offres et à la révision finale.)

Ensuite, nous avons déterminé le nombre total de points pour nos exigences, qui s'élève à 135 points (75 points pour le client lourd et 60 points pour le client léger). Nous avons divisé le nombre d'heures par le nombre de points pour obtenir une estimation du temps nécessaire par point.

Le calcul est le suivant :

540 heures / 135 points = 4 heures par point.

Pour être plus réalistes dans notre planification, nous avons ajouté un facteur de 40% à ce résultat. Ainsi, chaque point vaut **4 heures x 1,5**, ce qui donne **6 heures par point**. Nous avons ensuite multiplié les points de chaque fonctionnalité par ce résultat pour déterminer le nombre d'heures nécessaires par tâche, puis nous avons élaboré un échéancier avec une répartition équitable des heures pour chaque sprint, ce qui nous a permis d'établir notre temps requis (heures par personne).

Le temps de développement total est de **540 heures * 1.5 = 810 heures**

4.2 Échéancier du projet:

Sprint	Tâches/fonctionnalités	Temps requis [heures/ personne]	Date de début	Date de fin
Réponse à l'appel d'offres Phase 1 [2 semaines]	• SRS • Prototypes	40	26-08-2024	09-09-2024
Réponse à l'appel d'offres Phase 2 [2 semaines]	• Document d'architecture • Plan de projet • Prototypes	40	09-09-2024	23-09-2024
Réponse à l'appel d'offres Phase 2 [1 semaines]	• Document d'architecture • Plan de projet • Protocole de communication	20	23-09-2024	27-09-2024

Sprint 1 [2 semaines]	Client lourd	Clavardage - Intégration	30	418	28-09-2024	08-10-2024
		Clavardage - Canaux de discussion	24			
		Compte utilisateur et historique	18			
		Avatar	12			
		Modes de jeu	66			
		Vues d'administration et Vue de création (client lourd seulement)	60			
		Système de monnaie virtuelle	48			
	Client léger	Clavardage - Intégration	28			
		Clavardage - Canaux de discussion	24			
		Compte utilisateur et historique	12			
		Avatar	12			
		Modes de jeu	48			

		Système de monnaie virtuelle	36			
Sprint 2 [2 semaines]	Client lourd	Observateurs	36	288	09-10-2024	23-10-2024
		Personnalisation de l'application	24			
	Client léger	Observateurs	36			
		Personnalisation de l'application	24			
		Système d'amis	42			
	Exigences souhaitables	Ajout d'un historique détaillé des actions antérieures.	36			
		Mode de jeu Élimination rapide.	36			
		Défis de partie et indices	30			
		Système de blocage d'amis	24			
Sprint 3 [2 semaines]	Client lourd	Création de parties "amis seulement"	48	138	24-10-2024	07-11-2024
		Achievements	30			
	Client léger	Mode Hors-ligne	18			

		Lecture vocale des questions	12			
	Exigences souhaitables	IA correcteur des QRLS	30			
Sprint 4 [2 semaines]	Exigences souhaitables	Niveau/expérience	30	90	08-11-2024	22-11-2024
		Message audio/images	30			
		Classement général	30			
Sprint 5 [1 semaine]	Exigences souhaitables	Son de la tablette	6	24	22-11-2024	28-11-2024
		Soundboard	18			
Remise du produit final	Révision du code source version Mobile et Bureau			20	22-11-2024	29-11-2024
	Révision du plan de tests et résultats de tests			15	22-11-2024	29-11-2024
	Révision et préparation présentation oral			15	22-11-2024	29-11-2024
Total	Total des heures pour le projet			1 108	26-08-2024	29-11-2024

5. Équipe de développement

5.1. Mehdi El Harami:

- Étudiant avec 70 crédits en 3e année à l'École Polytechnique.
- Expérience en entreprise grâce à un stage en conseil IT réalisé l'été dernier.
- Participation à plusieurs projets universitaires et non universitaires pendant ses années à Poly.
- Possède une solide expérience en développement web, notamment avec les cadres Angular et Node.js.
- Maîtrise de plusieurs langages de programmation, dont C++, Python, JavaScript et TypeScript.
- Responsable du client lourd et gestion du compte AWS.

5.2 Arman Lidder:

- Étudiant de troisième année au baccalauréat en génie logiciel à Polytechnique Montréal.

- Expérience en développement web (Angular, Socket.IO, TypeScript, Express, HTML, CSS)
- Expérience en déploiement AWS EC2.
- Expérience en manipulation de bases de données avec MongoDB.
- Responsable du client lourd
- Gestionnaire du Gitlab

5.3. Hamza Boukaftane:

- Étudiant de troisième année au baccalauréat en génie logiciel à Polytechnique Montréal.
- Expérience en développement web (FastAPI, Django, Node.js, Socket.io, Angular, Streamlit, React, Express)
- Expérience avec les bases de données PostgreSQL et MongoDB
- Maîtrise des langages Python, JavaScript, TypeScript, Java, C/C++
- Expérience en gestion de projet lors d'un stage au CIUSSS de l'Est-de-l'île-de-Montréal.
- Développement et déploiement d'une application d'intelligence artificielle.
- Responsable du serveur et de l'échéancier

5.4 Maxime Pageot:

- Étudiant en troisième année de génie logiciel à Polytechnique Montréal.
- Expérience en développement web (React, Angular, Nodejs, Express, Flask)
- Maîtrise des langages Javascript et TypeScript
- Expérience avec les plateformes MongoDB et AWS
- Responsable serveur et de l'échéancier

5.5 Kasra Rahimi:

- Étudiant de troisième année en génie logiciel à Polytechnique Montréal.
- Expérience en développement d'applications web (Angular, NodeJS, Express, React, Socket.IO).
- Bonne compréhension des bases de données relationnelles (PostgreSQL) et de certaines bases de données non relationnelles (MongoDB).
- Excellente maîtrise de divers langages de programmation tels que Python, C++, Java, Javascript, Typescript, etc.
- Travaillé dans de nombreux projets de développement logiciel en équipe.
- Responsable du client léger et animateur du serveur Discord.

5.6 Mélissa Hammache:

- Étudiante de troisième année au baccalauréat en génie logiciel à Polytechnique Montréal.
- Expérience en développement web avec React, Angular et Node.js.

- Compétences en manipulation de bases de données : MongoDB, PostgreSQL.
- Maîtrise des langages de programmation : Python, JavaScript, TypeScript, Java.
- Participation à plusieurs projets académiques et personnels en développement logiciel.
- Responsable du client léger et de la gestion de la documentation du projet.

6. Entente contractuelle proposée

Cette proposition d'entente du système Poly Quiz dans le cadre du projet 3 contient plusieurs conditions formant ainsi l'ensemble des clauses de l'entente contractuelle proposée par l'équipe 101. L'École Polytechnique de Montréal et l'équipe 101 se sont mises d'accord sur les éléments suivants.

6.1 Type de contrat

L'équipe 101 a opté pour un contrat fixe couvrant l'ensemble des exigences essentielles spécifiées dans les documents de l'appel d'offres, ainsi qu'au moins 50 % des exigences souhaitables listées dans le document des exigences.

6.2 Respecter les échéances intermédiaires

L'équipe 101 doit finir le développement du système Poly Quiz en fonction des itérations décrites dans l'échéancier du projet.

6.3 Respect l'échéancier final

L'équipe 101 a l'obligation de livrer le système Poly Quiz, ainsi que le plan de tests et les résultats de ceux-ci au plus tard le 29 novembre 2024 à 23h59. Sinon, le client peut réduire le paiement de l'équipe d'un montant de 10% pour chaque jour de retard .

6.4 Paiement

L'équipe 101 propose un contrat fixe pour ce projet. Le client doit la payer en fonction de ces salaires mentionnés dans le document de l'appel d'offres initial :

- 110\$/heure pour le temps de développement
- 145\$/heure pour gérer le projet

En prenant en compte que l'équipe est composée de cinq développeurs et un gestionnaire de projet, que la charge de travail pour une personne de l'équipe est de $958h/5 = 192h$ (voir section 4.2) et que le temps de gestion équivaut à $100h + 50h = 150h$, le client s'engagera à déboursier la somme suivante pour couvrir les frais du projet :

$$5 * 110\$/h * 192h + 145\$/h * 150 = 105\,600\$ + 21\,750\$ = 127\,350\$$$

6.5 La négociation de tout changement

Le client peut demander des modifications des exigences à tout moment. Cependant, si l'équipe de développement estime que ces changements pourraient entraîner un retard dans la date de livraison, le client devra accorder à l'équipe 101 le temps supplémentaire requis pour les mettre en œuvre et couvrir les frais additionnels associés.